Vocabulary-NB4-TOEIC PRD

TOEIC 파트별 단어암기 학습 플랫폼. 사용자: iPhone/Android (모바일 앱). 관리자: 웹앱 + 데스크탑앱. 기본데이터: 각 파트 최소 200단어. 환경: 로컬 DB + 운영 DB 분리. 빌드/배포 분리 가능.

---

1. 개요

1.1 목적

TOEIC L&R 파트별 빈출 어휘를 IPA, 한글발음, 예문, 다국적 TTS 음원, 그림과 함께 학습하는 멀티디바이스 단어암기 서비스.

1.2 제품 범위

- **End-user 모바일 앱**: iPhone, Android (PWA/태블릿 반응형 포함).
- **Admin 웹앱**: 콘텐츠 관리자용 반응형 웹.
- **Admin 데스크탑앱**: 동일 코드 Electron/Tauri 래핑.
- **백엔드 API**: 인증, 학습, 진도, TTS, 콘텐츠 서빙.
- **DB 2종**: local (개발), prod (운영). 동일 스키마, 별도 마이그레이션 흐름.

1.3 핵심 기능

영역	기능
인증	Google / Apple / Kakao OAuth, Email + 비밀번호 + OTP
학습	파트별 단어카드, IPA, 한글발음, 예문, 그림, en-US/en-GB/en-AU TTS
운영	레슨 진행, SRS 복습큐, 진도율, 통계
설계	커리큘럼, 시험일 기반 일정관리
노트	개인 암기노트, 즐겨찾기, 태그
관리자	단어/예문/이미지/오디오 CRUD, TTS 배치, 사용자/통계

---

2. 기본 데이터 요건

2.1 파트별 최소 200단어

파트	유형	최소 단어수	주제 태그
Part 1	사진묘사	200	action/object/place/clothing/transport
Part 2	질의응답	200	wh-words, modals, time/place expressions
Part 3	대화	200	office, meeting, client, schedule
Part 4	담화	200	announcement, ad, news, voicemail
Part 5	문장완성	200	grammar collocation, business adj/adv
Part 6	지문완성	200	discourse marker, transition
Part 7	독해	200	contract, policy, email, notice
합계		**1,400+**	

2.2 단어 레코드 스키마

```
{
  "id": "toeic-p3-deadline",
  "part": 3,
```

```
  "word": "deadline",
  "pos": "noun",
  "meaning_ko": "마감일",
  "ipa": "/ˈdedlaɪn/",
  "pron_ko": "데드라인",
  "example_en": "We need to finish the report before the deadline.",
  "example_ko": "우리는 마감일 전에 보고서를 끝내야 한다.",
  "topic": ["office", "project"],
  "frequency_tier": 1,
  "image_prompt": "office team checking project deadline on calendar",
  "image_url": "https://cdn/.../deadline.webp",
  "synonyms": ["due date"],
  "distractors": ["outline", "receipt", "branch"],
  "audio": [
    {"locale":"en-US","provider":"azure","voice_id":"en-US-EmmaNeural","url":"..."},
    {"locale":"en-GB","provider":"polly","voice_id":"Amy","url":"..."},
    {"locale":"en-AU","provider":"azure","voice_id":"en-AU-NatashaNeural","url":"..."}
  ]
}
```

---

3. 디렉토리 구조

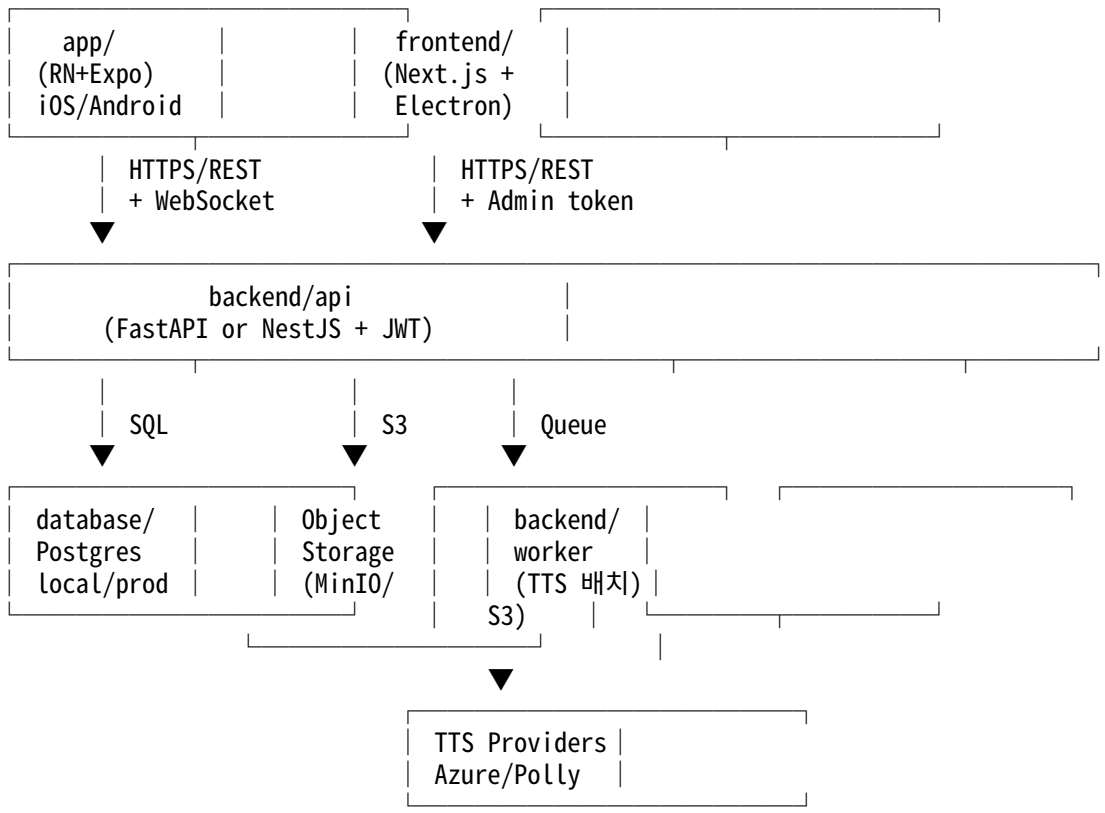
루트 5개 컴포넌트로 분리. 각 컴포넌트 독립 빌드/배포.

```
Vocabulary-NB4-TOEIC/
├── app/              # 사용자 모바일 앱 (iOS/Android)
│   ├── ios/
│   ├── android/
│   ├── src/
│   └── package.json
├── frontend/         # 관리자 웹앱 (+ 데스크탑 래퍼)
│   ├── web/          # Next.js admin
│   ├── desktop/      # Electron or Tauri shell
│   └── shared/       # 공통 컴포넌트/도메인 타입
├── backend/          # API 서버, 워커, TTS 파이프라인
│   ├── api/          # REST/GraphQL
│   ├── worker/       # TTS 배치, 이미지 처리
│   └── shared/       # 도메인 모델, DTO
├── database/         # 스키마, 마이그레이션, 시드
│   ├── migrations/
│   ├── seeds/        # 파트별 200단어 시드
│   ├── local/        # docker-compose용 로컬 DB 설정
│   └── prod/         # 운영 DB 설정/백업 스크립트
├── ops/              # 운영 설정
│   ├── docker/       # Dockerfile 모음
│   ├── compose/      # docker-compose.local.yml, .prod.yml
│   ├── k8s/          # 매니페스트 (선택)
│   ├── ci/           # GitHub Actions / GitLab CI
│   ├── scripts/      # 빌드/배포/마이그레이션 쉘
│   └── env/          # .env.local, .env.prod (gitignore, 템플릿만 커밋)
└── .doc/             # 기획/PRD/조사
```

---

4. 컴포넌트 연계 아키텍처

4.1 전체 데이터 흐름



ops/ orchestrates: docker-compose, CI/CD, env switching

4.2 컴포넌트별 책임

컴포넌트	책임	외부 의존
app	학습 UI, 오프라인 캐시, 푸시 수신	backend/api, OAuth providers
frontend	콘텐츠 CRUD, 사용자/통계, 배치 트리거	backend/api, Object Storage
backend/api	인증, 학습/진도 로직, 콘텐츠 서빙	database, storage, worker queue
backend/worker	TTS 생성, 이미지 변환, 시드 임포트	TTS API, storage, database
database	단어/사용자/진도/일정 영속화	-
ops	빌드, 컨테이너, 환경 전환, 배포	Docker, CI runner

4.3 환경 분리

- `.env.local` → database/`local` (Postgres docker) + backend (dev mode) + `frontend` (dev server) + app (Expo dev client).
- `.env.prod` → 관리형 Postgres + S3 + CDN + Expo EAS prod 빌드 + Next.js prod 배포.
- `ops/compose/docker-compose.local.yml`: 로컬 풀스택 부트스트랩.
- `ops/compose/docker-compose.prod.yml`: 운영 풀스택 (또는 K8s/PaaS 대체).
- `ops/scripts/migrate.sh --env=local|prod`: 마이그레이션 분기.

---

5. 오픈소스 비교 및 추천

5.1 app/ — 사용자 모바일

항목	React Native + Expo	Flutter	Capacitor + React
라이선스	MIT	BSD-3-Clause	MIT
언어	TypeScript	Dart	TypeScript
렌더링	Native bridge (New Arch)	Self-rendered (Impeller/Skia)	WebView
iOS/Android	O/O	O/O	O/O
태블릿 반응형	O (Flexbox + Dimensions)	O (Adaptive)	O (CSS)
오디오 (TTS 재생)	Expo AV, react-native-track-player	just_audio	Capacitor Media
푸시	Expo Notifications	firebase_messaging	Capacitor Push
OTA 업데이트	EAS Update O	Shorebird (3rd party)	Capgo
웹/Admin 코드 공유	높음 (RN-web)	낮음	최고
채용/생태계	매우 큼 (npm)	큼 (pub.dev)	큼
추천 우선순위	**1위**	2위	3위

추천: React Native + Expo (MIT). - 이유: TypeScript 단일 스택으로 frontend/backend와 통일. EAS로 빌드/OTA 일원화. 학습 카드/오디오/푸시 패키지 성숙. - 보조 라이브러리: - 라우팅: **Expo Router** (MIT) - 상태: **Zustand** (MIT), 서버상태 **TanStack Query** (MIT) - 폼: **React Hook Form** (MIT) + **Zod** (MIT) - UI: **NativeWind** (MIT) 또는 **Tamagui** (MIT) - 오디오: **react-native-track-player** (Apache-2.0) — 재생큐/lock-screen 컨트롤 - 로컬 DB: **WatermelonDB** (MIT) 또는 **expo-sqlite** (MIT) - KV 캐시: **react-native-mmkv** (MIT) - 오류추적: **Sentry RN** (MIT) - 분석: **PostHog RN** (MIT)

5.2 frontend/ — 관리자 웹앱 + 데스크탑

5.2.1 웹앱 프레임워크

항목	Next.js	Remix	Refine
라이선스	MIT	MIT	MIT
용도	풀스택 React	풀스택 React	Admin/CRUD 전용
App Router/RSC	O	-	O (Next 위)
어드민 빠른 구축	직접 구현	직접 구현	매우 빠름
추천	**메인 관리자**	대안	보조 (빠른 CRUD 화면)

추천: Next.js (MIT) 메인 + **Refine (MIT)** 보조.

5.2.2 관리자 UI 컴포넌트

항목	shadcn/ui	Ant Design	MUI
라이선스	MIT	MIT	MIT
스타일	Tailwind 기반, 코드 소유	종합 디자인시스템	Material
어드민 적합	매우 좋음	매우 좋음 (Table/Form)	좋음
추천	**1위**	2위 (대량 테이블)	3위

5.2.3 보조 CMS / 운영툴

항목	Directus	Strapi	Appsmith	ToolJet
라이선스	BSL 1.1 (소형 무료)	SSPL/Enterprise	Apache-2.0	AGPL-3.0
역할	DB-first CMS	Headless CMS	내부툴 로우코드	내부툴 로우코드
Postgres 연결	O (자동 스키마 미러)	O (자체 모델)	O	O
콘텐츠 워크플로우	매우 강함	강함	보통	보통
추천	**콘텐츠/미디어 CMS**	대안	운영 대시보드	빠른 보조툴

전략: - 단어/예문/이미지/오디오 CRUD → **Directus** (BSL). - 커리큘럼/레슨 편집기, 통계, 승인 워크플로우 → **Next.js 커스텀** (MIT). - CS/임시 운영 패널 → **Appsmith** (Apache-2.0).

5.2.4 데스크탑 래퍼

항목	Electron	Tauri
라이선스	MIT	MIT/Apache-2.0
런타임	Chromium + Node	OS WebView + Rust
바이너리 크기	100MB+	5-20MB
보안	보통	강함
추천	안정/생태계 우선 시	**1위** (경량, 보안)

추천: Tauri (MIT/Apache-2.0). - 이유: Next.js admin을 그대로 패키징. 경량 배포. Windows/macOS/Linux 동시.

5.3 backend/ — API + Worker

5.3.1 API 프레임워크

항목	FastAPI	NestJS	Spring Boot
라이선스	MIT	MIT	Apache-2.0
언어	Python	TypeScript	Java/Kotlin
비동기 I/O	O (asyncio)	O	O (WebFlux)
자동 OpenAPI	O	O (Swagger)	O
TTS/배치 친화	매우 좋음 (numpy/audio)	좋음	보통
팀 통일성 (frontend TS)	별개 언어	**동일 TS**	별개
추천	**1위** (TTS/AI 친화)	2위 (TS 통일)	3위

추천: FastAPI (MIT). - 이유: TTS/이미지/AI 배치는 Python 생태계가 압도적. OpenAPI 자동 생성으로 RN/Next 클라이언트 SDK 즉시 생성. - 보조: **Pydantic v2** (MIT), **SQLAlchemy 2** (MIT), **Alembic** (MIT) 마이그레이션, **uvicorn/gunicorn** (BSD/MIT).

5.3.2 인증

항목	Keycloak	Authelia	Supabase Auth	Authentik
라이선스	Apache-2.0	Apache-2.0	Apache-2.0	MIT
OIDC/OAuth2	O	O	O	O
Google/Apple	O	O	O	O
Kakao	OIDC 커스텀	OIDC 커스텀	OIDC 커스텀	OIDC 커스텀
이메일 OTP	O	O	O	O
Self-host 난이도	중상	중	쉬움 (BaaS)	중
추천	기업급 SSO	경량 SSO	**1위 (속도)**	**2위 (UI 우수)**

추천: Supabase Auth (Apache-2.0) 또는 Authentik (MIT). - MVP: Supabase Auth (self-host 또는 managed). Postgres 직접 연결 가능. - 장기 자체호스팅 강화: Authentik.

5.3.3 Worker / Queue

항목	Celery	RQ	Dramatiq	Temporal
라이선스	BSD-3	BSD	LGPL-3.0	MIT
브로커	Redis/RabbitMQ	Redis	Redis/RabbitMQ	자체
워크플로우	보통	약함	보통	매우 강함
TTS 배치 적합	매우 좋음	좋음	좋음	좋음
추천	**1위**	2위	3위	복잡 워크플로우 시

추천: Celery (BSD-3) + Redis (BSD-3 / RSAL 7.4+ 주의). - Redis 대체: **Valkey** (BSD-3, Linux Foundation 포크) — Redis 라이선스 회피.

5.3.4 TTS Provider 어댑터

Provider	라이선스/모델	en-US	en-GB	en-AU	비용 모델
Azure Speech	상용 SaaS	다수 Neural	다수 Neural	다수 Neural	글자 단위
AWS Polly	상용 SaaS	Joanna/Matthew 등	Amy/Brian 등	Olivia/Nicole 등	글자 단위
Google Cloud TTS	상용 SaaS	다수	다수	다수	글자 단위
Coqui TTS (오픈소스)	MPL-2.0	O (모델 따라)	△	△	self-host
Piper	MIT	O	O	△	self-host (경량)

추천: Azure Speech 메인 + AWS Polly 백업 (멀티벤더). - 자체호스팅 옵션: **Piper (MIT)** — 비용 0, 품질은 Neural 대비 낮음. 학습/스테이징용.

5.4 database/ — DB 엔진 + 마이그레이션

5.4.1 DB 엔진

항목	PostgreSQL	MySQL/MariaDB	SQLite
라이선스	PostgreSQL License (BSD류)	GPL-2.0 / GPL-2.0	Public Domain
분석 쿼리	매우 강함	좋음	약함
JSON	JSONB 강력	JSON 보통	JSON 보통
풀텍스트	tsvector	O	FTS5
운영 적합	**1위**	2위	로컬 캐시용

추천: PostgreSQL (PostgreSQL License). - 확장: **pgvector** (PostgreSQL License) — 단어/예문 유사도, AI 추천. - 로컬 캐시 (앱 내): **SQLite** (PD).

5.4.2 마이그레이션 도구

항목	Alembic	Flyway	Liquibase	Prisma Migrate
라이선스	MIT	Apache-2.0	Apache-2.0	Apache-2.0
언어 친화	Python	범용	범용 (XML/YAML)	TS/Node
자동 diff	약함 (manual)	약함	약함	강함
환경 분기	env.py 분기	프로파일	컨텍스트	env 분기

추천: Alembic (MIT) (FastAPI/SQLAlchemy 동반). - `database/migrations/env.py`에서 `ENV=local|prod` 분기.

5.4.3 오브젝트 스토리지

항목	MinIO	SeaweedFS	Garage
라이선스	AGPL-3.0	Apache-2.0	AGPL-3.0
S3 호환	O	O	O
운영 단순성	매우 좋음	보통	좋음
추천 (로컬)	**1위**	2위	3위
운영	AWS S3 / Cloudflare R2		

추천: - 로컬: **MinIO (AGPL-3.0)** in docker-compose. - 운영: **AWS S3** 또는 **Cloudflare R2** (egress 0원, 비용 우위).

5.5 ops/ — 운영

5.5.1 컨테이너 / 오케스트레이션

항목	Docker + Compose	Podman	Kubernetes (k3s)
라이선스	Apache-2.0	Apache-2.0	Apache-2.0
로컬 개발	**1위**	2위	과함
운영 scale	단일 호스트	단일 호스트	멀티노드

추천: - 로컬: **Docker + Compose** (Apache-2.0). - 운영 초기: 단일 VM + Compose, 트래픽 증가 시 **k3s** (Apache-2.0).

5.5.2 CI/CD

항목	GitHub Actions	GitLab CI	Drone	Woodpecker
라이선스	상용 (무료 한도)	MIT / EE 상용	Apache-2.0	Apache-2.0
모바일 빌드	EAS 연동 쉬움	가능	가능	가능

추천: GitHub Actions (GitHub 호스팅 시) + **Expo EAS** (모바일 빌드 자동화).

5.5.3 리버스 프록시 / TLS

항목	Traefik	Caddy	Nginx
라이선스	MIT	Apache-2.0	BSD-2
자동 TLS	O	**O (default)**	수동/스크립트
Docker 자동 검출	O	△	△

추천: Caddy (Apache-2.0) — 자동 HTTPS, 설정 단순.

5.5.4 모니터링

항목	Prometheus + Grafana	Loki	Sentry (self-host)	OpenTelemetry
라이선스	Apache-2.0 / AGPL-3.0	AGPL-3.0	BSL 1.1	Apache-2.0
역할	메트릭	로그	오류추적	표준 계측

추천 스택: Prometheus + Grafana + Loki + Sentry self-host + OpenTelemetry SDK.

---

6. 빌드 / 배포 시나리오

6.1 로컬 부트스트랩

```
# 1) DB + storage + cache 띄움
cd ops/compose && docker compose -f docker-compose.local.yml up -d

# 2) 마이그레이션 + 시드 (파트별 200단어)
cd ../../database && ENV=local ../ops/scripts/migrate.sh
ENV=local ../ops/scripts/seed.sh

# 3) backend dev
cd ../backend/api && uvicorn main:app --reload

# 4) frontend admin dev
cd ../../frontend/web && pnpm dev

# 5) app dev
cd ../../app && pnpm expo start
```

6.2 운영 배포

- `git push origin main` → GitHub Actions.
- Job 1: backend image build & push → ECR/GHCR.
- Job 2: frontend Next.js Vercel/self-host 배포.
- Job 3: app EAS build (iOS/Android) → TestFlight/Play Internal.
- Job 4: `ENV=prod` Alembic 마이그레이션 (수동 승인 게이트).
- Job 5: Tauri admin desktop 빌드 → GitHub Releases.

6.3 환경 변수 매트릭스

키	local	prod
`DATABASE_URL`	`postgres://localhost:5432/toeic_local`	`postgres://prod-host/toeic_prod`
`S3_ENDPOINT`	`http://minio:9000`	`https://s3.amazonaws.com`
`TTS_PROVIDER`	`piper` (자체)	`azure`
`AUTH_REDIRECT`	`http://localhost:3000`	`https://admin.example.com`
`SENTRY_DSN`	empty	prod DSN

---

7. 비기능 요건

항목	요건
가용성	운영 99.5% 목표
응답시간	API p95 < 300ms
오디오 지연	첫 재생 < 1s (CDN 캐시 기준)
오프라인	학습 카드 24h 캐시
보안	OAuth2/OIDC, JWT, HTTPS only, RLS/Row policy, 비밀번호 Argon2
개인정보	한국 PIPA / GDPR 대비, 학습 로그 익명화 옵션
접근성	WCAG 2.1 AA 목표
i18n	ko (기본), en 확장 가능
라이선스 호환	모든 임베디드 오픈소스 라이선스 NOTICE 자동 생성 (ops/scripts/license-report.sh)

8. 라이선스 요약 (임베디드 핵심)

컴포넌트	핵심 라이브러리	라이선스	상업적 배포
app	React Native, Expo, Zustand, TanStack Query, NativeWind, Track Player	MIT / Apache-2.0	가능
frontend	Next.js, shadcn/ui, Refine, Tauri	MIT / Apache-2.0	가능
frontend (CMS)	Directus	BSL 1.1	매출 한도 내 무료, 초과 시 상용
backend	FastAPI, SQLAlchemy, Alembic, Celery	MIT / BSD	가능
backend (auth)	Supabase Auth / Authentik	Apache-2.0 / MIT	가능
backend (TTS OSS)	Piper / Coqui	MIT / MPL-2.0	가능 (Coqui는 모델 라이선스 확인)
database	PostgreSQL, pgvector, SQLite	PostgreSQL/PD	가능
storage	MinIO	AGPL-3.0	self-host OK, 수정배포 시 소스공개
ops	Docker, Caddy, Prometheus, Grafana	Apache-2.0 / AGPL-3.0	Grafana AGPL 주의
ops (오류)	Sentry self-host	BSL 1.1	비-경쟁 용도 사용 가능

AGPL/BSL 컴포넌트는 **수정/재배포** 시 라이선스 조건 발생. self-host로만 사용하면 대부분 영향 없음. 최종 결정 전 법무 검토 권장.

---

9. 단계별 로드맵

단계	산출물
M0 (셋업)	디렉토리 5종 골격, docker-compose.local, Alembic init, Next.js admin 골격, Expo 골격
M1 (데이터)	파트별 200단어 시드 7세트 + IPA/한글발음/예문/그림 prompt
M2 (TTS)	en-US/en-GB/en-AU 배치 파이프라인, CDN 업로드
M3 (사용자 앱 MVP)	로그인, 레슨 커버, 카드, TTS 재생, 진도 저장
M4 (관리자)	단어 CRUD (Directus 연결), 레슨 편집기 (Next 커스텀)
M5 (학습설계)	커리큘럼, SRS 큐, 시험일 기반 일정
M6 (운영화)	통계, 모니터링, OTA, 데스크탑 Tauri 빌드

---

10. 미결 의사결정

항목	후보	결정 필요 시점
모바일 프레임워크	RN+Expo vs Flutter	M0 시작 전

항목	후보	결정 필요 시점
백엔드 언어	FastAPI(Python) vs NestJS(TS)	M0 시작 전
인증	Supabase Auth vs Authentik	M1
TTS 운영	Azure vs Polly vs 듀얼	M2
CMS 도입 여부	Directus 사용 / Next 단일	M4
데스크탑	Tauri vs Electron	M6

끝.